

УДК 004.724, 004.725, 004.052.3, 519.687.7

А. А. Бегаев¹, А. Н. Сальников²

МЕТОД ИЗМЕРЕНИЯ ЗАДЕРЖЕК ПРИ ПЕРЕДАЧЕ ДАННЫХ МЕЖДУ ГРАФИЧЕСКИМИ ПРОЦЕССОРАМИ, НАХОДЯЩИМИСЯ НА РАЗНЫХ УЗЛАХ ВЫЧИСЛИТЕЛЬНОГО КЛАСТЕРА*

В данной статье предложены и реализованы в виде программного кода принципы нагрузочно-го тестирования вычислительного кластера с большим количеством GPU (Graphics Processing Unit), распределенным по узлам кластера. В результате собирается информация о задержках во времени при передаче данных разного размера между всеми GPU в системе. Разработано два режима тестирования “all to all” и “one to one”. В первом режиме все GPU передают данные всем GPU одновременно. Во втором режиме в один момент времени происходит передача только между двумя GPU. По результатам тестирования, полученным с вычислительного кластера K60 (ИПМ РАН), было показано, что коммутационная среда суперкомпьютера при передаче информации между GPU не является однородной не только для передачи через сеть, но и для GPU в рамках одного узла вычислительного кластера.

Ключевые слова: нагрузочное тестирование, коммуникационная среда, MPI, GPU.

1. Введение. В настоящее время суперкомпьютеры активно используются для решения различных прикладных задач, связанных с тяжелыми многостадийными расчетами и обработкой больших объемов информации. Например, это могут быть области, связанные с расчетом климатических моделей [1], получением изображений от большого количества источников и машинным обучением [2]. В процессе решения этих задач часто приходится применять однотипные операции к различным однородным данным. Появление в суперкомпьютере специальных вычислителей, таких, как GPU (Graphics Processing Unit), привели к появлению высоко-параллельной парадигмы, когда большая параллельная задача может быть разбита на несколько меньших, одновременно запускающихся на вычислителях разного класса [3].

Основным преимуществом GPU перед многоядерным процессором является поддержка огромного количества легковесных потоков. Наборы таких потоков применяют одни и те же последовательности операций к данным параллельно. За счет того, что таких потоков и наборов много, достигается высокая параллельность вычислений, это позволяет создавать эффективную реализацию вычислений на GPU [4]. Суперкомпьютер состоит из большого количества узлов, каждый из которых может обладать несколькими GPU. Примером такой системы является суперкомпьютер Summit [5] (первое место в top500 (июнь 2019)). Параллельная программа позволяет задействовать в процессе исполнения огромное количество узлов и GPU. Однако, появление дополнительных элементов в архитектуре приводит к усложнению коммуникации из-за необходимости передачи данных от одного независимого вычислителя к другому. Устройства GPU, в случае, когда параллельной программе необходимо передавать между ними данные, могут находиться на разных узлах вычислительного кластера.

Размер данных, которые требуется передать между узлами и GPU, может быть довольно большим. В данной статье показан метод оценки затрат на передачу для трехмерных уравнений Больцмана [6]. При неправильном распределении вычислений в параллельной программе может возникнуть ситуация, когда время, затраченное на вычисления, будет гораздо меньше, нежели время передачи данных от вычислителя к вычислителю. Память GPU является независимой от

¹ Факультет ВМК МГУ, студ., e-mail: artur.begaev@cs.msu.ru

² Факультет ВМК МГУ, в.н.с., к.ф.-м.н, e-mail: salnikov@cs.msu.ru

* Работа частично поддержана грантом РФФИ проект 17-07-01562 и научной темой НИОКТР АААА-А16-116021510039-7.

памяти, доступной процессорам на узле [7]. Соответственно, при передаче данных необходимо учитывать возникающие задержки внутренней сети суперкомпьютера, состоящей, при усложненной архитектуре, не только из коммутаторов, сетевых карт и маршрутизаторов, но и шины, к которой на узлах подключены GPU-устройства.

Таким образом, программистами накладывается требование на оптимизацию передач данных между компонентами вычислительной системы. Сведения о задержках при передаче данных также необходимы системным администраторам вычислительных кластеров для выявления причин уменьшения пропускной способности внутренней сети суперкомпьютера и своевременного устранения неполадок.

Для передачи данных между узлами суперкомпьютера на прикладном уровне чаще всего используется стандарт MPI (Message Passing Interface) [8]. Работа с GPU, в основном, производится при помощи специальной платформы CUDA [9] и сопутствующих библиотек, разработанных компанией Nvidia. Так как MPI и CUDA являются не зависимыми друг от друга, необходимо правильно использовать их вместе для обеспечения возможности написания эффективной параллельной программы, существенно использующей доступный потенциал вычислительного кластера.

В данной работе предлагается измерить задержки при передаче данных различного размера между набором GPU в вычислительном кластере, используя средства передачи данных MPI, — для взаимодействия между узлами кластера, и CUDA — для взаимодействия с GPU-устройством. Для этого предлагается реализовать тесты, измеряющие задержки при передаче данных, в следующих режимах:

- one to one – фиксируется 2 GPU в узлах вычислительного кластера, доступных программе, передача в один момент времени может происходить только между ними двумя;
- all to all – происходит одновременная передача от всех GPU, доступных программе на вычислительном кластере ко всем GPU, доступным программе.

Введем некоторые понятия: иницилирующую передачу GPU будем называть GPU-источник, а принимающую сообщения — GPU-приемник. Данные передаются при этом не только от одной GPU внутри узла к другой, но и между узлами кластера.

Данная статья посвящена сбору данных о состоянии коммутационной среды суперкомпьютера, с учетом передач между GPU. Собранные данные в дальнейшем позволят провести анализ этой среды. Анализ производится для того, чтобы определить, существуют ли неверно функционирующие устройства в системе, а для нахождения лучшего варианта схемы распределения параллельной программы по вычислительной системе с учетом GPU. Свои подходы к решению похожих проблем предлагают исследователи в статьях, рассмотренных далее.

2. Обзор методов сбора сведений о задержках. В работе [10] авторами была предложена идея мониторинга внутренней сети суперкомпьютера, состоящей только из коммутаторов, маршрутизаторов и сетевых карт, соединенных кабелями, посредством нагрузочного тестирования. Исследователи реализовали большое количество различных схем передачи данных между узлами: один к одному, все ко всем и пр. Также реализовано специальное приложение, позволяющее анализировать собранную информацию о задержках, посредством визуализации. При помощи программного обеспечения, описанного в работе, авторы предлагают находить неполадки во внутренней сети суперкомпьютера, обнаруживать различные особенности коммуникаций и узлов. Данная работа позволяет обнаруживать некоторые особенности поведения сети суперкомпьютера при помощи наборов различных тестов. Однако, авторы не учитывают присутствие GPU в распределенной системе. Устройства GPU в вычислительном кластере могут обмениваться посредством сети либо шины, за счет чего характер задержек может измениться, что важно при распределении вычислений по нескольким GPU.

Статья [11] посвящена поиску “бутылочных горлышек” во внутренней сети суперкомпьютера. Авторами предлагается в течение некоторого времени с равными промежутками отправлять между узлами сообщения фиксированной длины и измерять длительность передач. В дальнейшем они обрабатывают полученные данные и производят кластеризацию алгоритмом K-Means.

Благодаря этому они находят узлы суперкомпьютера, между которыми поведение возникающих задержек является одинаковым. Используя эту информацию, исследователи в своей работе предлагают находить “выбивающиеся” узлы из каких-либо кластеров, и считать их проблемными. Они разработали специальное приложение, позволяющее на основе собранных в результате тестирования данных, производить мониторинг внутренней сети суперкомпьютера. Авторы данной статьи показывают, что для конкретной длины сообщения можно обнаруживать “узкие места” в сети суперкомпьютера, но они не учитывают того, что с изменением размеров передаваемых данных схема маршрутизации и алгоритмы перегрузки могут меняться [12]. Таким образом, для получения полной картины о состоянии сети суперкомпьютера требуется проводить несколько тестирований, что требует ресурсов и времени. Также, в данной работе не учитываются передачи данных между GPU. Их следует учитывать, так как из-за наличия GPU изменяется поведение задержек в вычислительном кластере.

В [13] исследователи предложили метод оптимизации передачи информации в суперкомпьютере между GPU. Основной идеей работы является использование многопоточного приложения, где каждый поток контролирует какую-либо из видеокарт. Основной проблемой систем с несколькими GPU является то, что в один момент времени одну GPU может контролировать только один “физический” процессорный поток. Главной сложностью является правильное распараллеливание приложения по этим потокам, поэтому многие разработчики используют один процессорный поток для управления всеми GPU. В результатах своего исследования авторы показывают, что если использовать для каждой GPU отдельный процессорный поток, тогда значительно увеличивается производительность приложения. Своей статьей авторы показывают одну из возможностей увеличения скорости вычислений в распределенной системе, однако они не учитывают особенности коммутационной среды суперкомпьютера, предполагая ее однородной. В реальном суперкомпьютере некоторые компоненты могут работать некорректно, поэтому следует учитывать состояние внутренней сети.

3. Прделанные действия. Для создания программной системы было сделано следующее:

- разработан алгоритм тестирования и сбора информации о задержках при передаче данных между GPU для режимов all to all и one to one; режимы тестирования реализованы на языке C, для передачи данных используются MPI и CUDA;
- код внедрен в существующий проект `network_test2` [14];
- сохранена совместимость с данными получаемыми `network_test2`, для этого результаты записываются с использованием `NetCDF4` [15].

В качестве информации о задержках предложено использовать следующие статистики, полученные в результате многих итераций тестирования: среднее, медиану, стандартное отклонение и минимум.

4. Описание программной реализации. Одной из проблем при реализации тестирующего приложения является то, что при запуске параллельной программы неизвестно количество GPU, которые доступны для каждого из MPI-процессов. Также, существует проблема, связанная со схемой реализации очереди постановки задач на суперкомпьютере: на одну GPU может приходиться несколько MPI-процессов. При этом возникает конкуренция за управление GPU между такими процессами. Однако, CUDA запрещает поведение, когда много процессов одновременно используют GPU, предоставляя доступ только одному из процессов. Поэтому для реализованных схем тестирования накладывается следующее ограничение: требуется, чтобы на одну GPU приходилось не более одного MPI-процесса. Идеальной ситуацией является, когда один MPI-процесс соответствует физическому узлу суперкомпьютера.

Для решения проблемы с неизвестным количеством GPU, перед измерением задержек, выполняется предварительная процедура обнаружения GPU. При помощи вызова `cudaGetDeviceCount` можно определить количество доступных процессу устройств. Однако, вследствие распределенности системы и независимости процессов друг от друга, необходимо произвести обмен инфор-

мацией о количестве видеокарт. Это происходит следующим образом: после того, как все процессы узнают о количестве видеокарт, они передают собранную информацию процессу с рангом 0 (мастер-процесс).

После этого каждый процесс получает информацию о GPU, доступных ему. Это происходит при помощи функции *cudaGetDeviceProperties*. На вход данной функции передается указатель на специальную структуру *cudaDeviceProp*. Вызовы данных процедур производятся для каждой GPU. Из каждой структуры извлекается имя соответствующей GPU. К каждому имени добавляется имя узла, на котором запущен процесс, полученное при помощи системного вызова *gethostname*. Полученные имена передаются мастер-процессу, которые в дальнейшем сохраняются в специальный файл *hosts.txt*. Каждая строка данного файла представляет имя GPU в системе. Формат строки выглядит следующим образом: <имя узла>--<имя GPU>.

Также из-за того, что система, в которой подразумевается запуск тестирующего приложения, является распределенной и в ней отсутствуют глобальные часы, требуется определить, что будет считаться временем (задержкой), потраченным на передачу сообщения. В данной работе предполагается, что потраченное время будет учитываться только на прием сообщения, т.е. на GPU-приемнике.

Узнав общую конфигурацию системы, каждый процесс входит в один из режимов тестирования, предварительно заданный пользователем в аргументах перед запуском тестирующего приложения. Для режима one to one для GPU следует указывать ключ *one_to_one_cuda*, а для режима all to all для GPU следует указывать ключ *all_to_all_cuda*. Также при помощи ключей *-b*, *-e*, *-s*, *-n* можно указывать начальный размер сообщения, конечный размер сообщения, шаг изменения длины сообщения и количество замеров соответственно. Мастер-процесс также создает выходные файлы, каждый из которых содержит следующую информацию:

- трехмерный куб, измерения которого соответствуют номеру GPU-источника, номеру GPU-приемника и длине сообщения, для которой производилось измерение;
- режим тестирования;
- начальная длина сообщения;
- конечная длина сообщения;
- шаг изменения длины сообщения;
- количество итераций.

Перед проведением тестирования каждый процесс создает специальный массив, который в дальнейшем будет содержать собранные статистики о моментах приема сообщений от всех GPU в системе, GPU, которые доступны данному процессу. Тестирование происходит для каждой длины сообщения, которое может получиться с указанными ранее параметрами указанное количество раз (итераций). После проведения всех итераций тестирования происходит вычисление статистик. Средняя задержка передачи между видеокартой-источником i и видеокартой-приемником j считается по следующей формуле: $\mu^{ij} = \frac{1}{N_{iter}} \sum_{k=1}^{N_{iter}} d_k^{ij}$. Стандартное отклонение

$\sigma = \sqrt{\frac{1}{N_{iter}} \sum_{k=1}^{N_{iter}} (d_k^{ij} - \mu^{ij})^2}$, где N_{iter} — количество итераций тестирования, d_k^{ij} — задержка, полученная при передаче от GPU i до GPU j в итерации k . Для получения медианы m сортируется полученный ряд задержек и берется центральный элемент этого ряда.

После проведения тестирования и вычислений статистик, все процессы передают собранную информацию мастер-процессу посредством MPI. Мастер сохраняет информацию о задержках для каждой длины сообщения. Далее рассмотрим логику работы реализованных тестов для GPU.

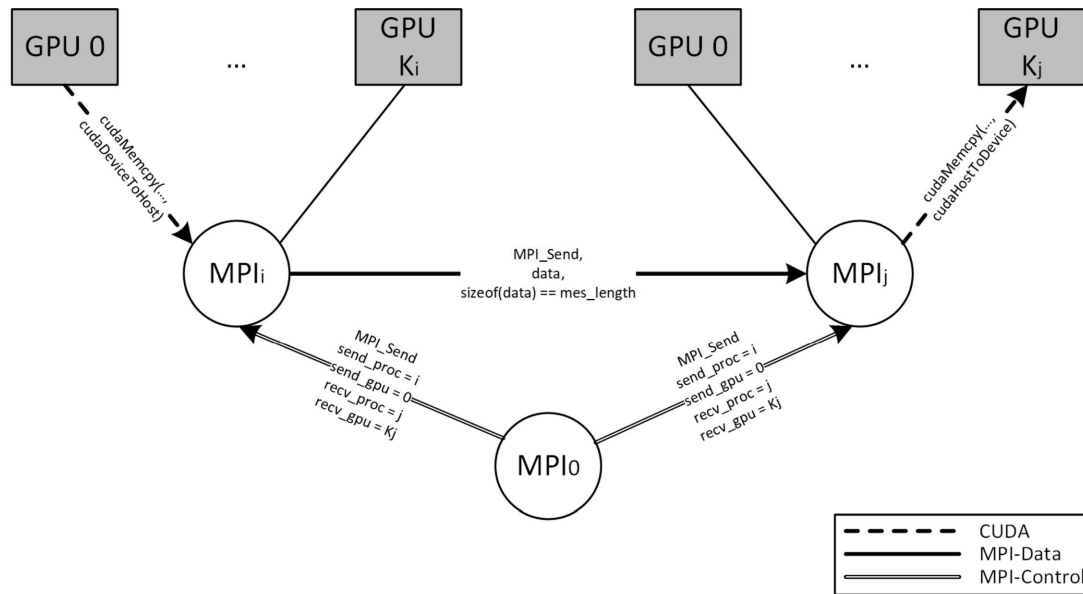


Рис. 2. Схема передачи данных между GPU на разных узлах

с аргументами *start* и *stop*. Вычисленное время имеет погрешность в 0.5 микросекунд, и выражается в миллисекундах. Оно сохраняется в соответствующее место в упомянутом ранее массиве.

Если обнаружилось, что видеокарты находятся на разных процессах, то процесс передачи информации между ними производится и средствами CUDA, и средствами MPI. На рис. 2 показана данная схема. Пунктирной линией обозначены передачи через шину, сплошной — через сеть суперкомпьютера, двойной — отправка контролирующего запроса от мастер-процесса. Рассмотрим процедуры, которые запускаются на процессах, имеющих GPU-источник и GPU-приемник. Для обоих процессов выставляется активная видеокарта посредством функции *cudaSetDevice*, номер видеокарты зависит от того, является ли процесс обладателем GPU-источник либо GPU-приемник. Оба процесса затем выделяют память под данные посредством *cudaMalloc* на соответствующих видеокартах. Если процесс оказался “источником”, соответственно активная GPU будет отправлять данные, то первым делом производится копирование с GPU на CPU при помощи *cudaMemcpy*, с параметром *cudaDeviceToHost*. Затем, переданные данные отправляются при помощи MPI через функцию *MPI_Send*. В то же время, процесс с GPU-приемником ожидает сообщения от другого процесса при помощи *MPI_Recv*. После получения сообщения оно копируется на видеокарту с использованием *cudaMemcpy* с параметром *cudaHostToDevice*. До начала блокирующего ожидания получения сообщения (*MPI_Recv*) происходит запись о времени начала передачи при помощи вызова *gettimeofday* в переменную *start*. Временем *stop* окончания передачи предполагается момент завершения операции синхронного копирования на GPU — *cudaMemcpy*. Временем, затраченным на передачу, в итоге, считается время *stop – start*.

4.2. Режим all to all. В данном режиме передачи данных все процессы считаются равнозначными друг другу. Каждый процесс инициализирует передачу информации от каждой своей GPU до каждой GPU, принадлежащей другому процессу. Все передачи происходят асинхронно. Схема реализации передач приведена на рис. 3. Пунктирные линии обозначают асинхронные передачи через шину. В зависимости от направленности стрелки передача происходит напрямую между GPU, либо с GPU на процессор, где запущен процесс, либо в обратную сторону. Асинхронная передача между процессами (узлами) происходит посредством MPI (сети) (на рис. 3 обозначена пунктирной с двумя точками линией). Для такой передачи вводится специальная сущность — глобальный ранг GPU. Такая сущность может быть введена по причине фиксированности номеров MPI-процессов и порядка GPU-устройств на шине, тем самым возможно задать порядковый номер GPU во всей системе. Процесс на узле кластера может монополюльно занимать GPU-устройство. Для корректного тестирования системы данным тестом требуется назначать по

одному MPI-процессу на каждый узел суперкомпьютера. В дальнейшем, для упрощения обозначим количество видеокарт, доступных узлу i , через V_i , общее количество видеокарт в системе — через V_{sys} .

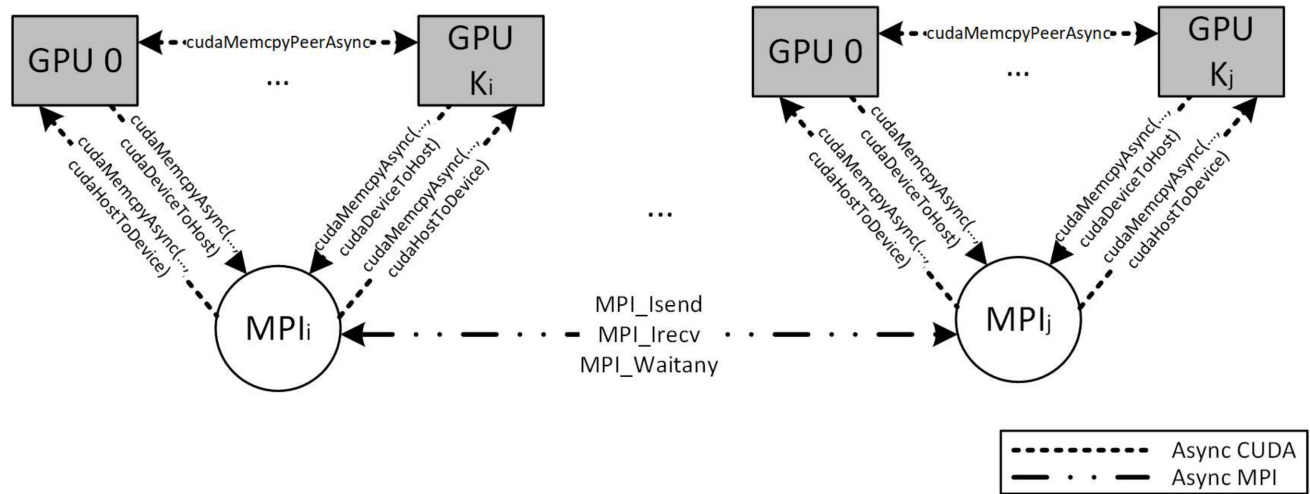


Рис. 3. Схема передачи данных между GPU в режиме all to all

Предварительно на каждой видеокарте инициализируются объем памяти для передачи на все видеокарты системе и объем памяти для приема информации от всех видеокарт посредством *cudaMalloc*. Также, предварительно создаются сущности *cudaEvent* в количестве $V_i \cdot V_i$, потому что для передачи данных между GPU на одном узле используется иная схема, нежели для передачи данных на другой узел. Также для каждой GPU создается $V_{sys} \cdot V_{sys}$ потоков, с помощью которых будет производиться передача данных с/на видеокарты.

В специальном цикле каждый процесс смотрит, какому процессу принадлежит GPU-приемник. В случае, если видеокарты принадлежат одному процессу, то передачи происходят аналогично режиму one to one. Создается прямой доступ от видеокарты до видеокарты на процессе (узле) через шину с помощью вызова *cudaEnablePeerAccess*. Перед началом передачи данных происходит запись в соответствующий *cudaEvent* $start^{ij}$, где i — номер GPU-источника, j — номер GPU-приемника. Затем инициализируется неблокирующее копирование *cudaMemcpyPeerAsync*. После завершения этого произойдет запись в *cudaEvent* $stop^{ij}$. Если видеокарты находятся на разных узлах, то инициализируется асинхронное копирование соответствующих данных с видеокарты на CPU при помощи *cudaMemcpyAsync* с параметром *cudaHostToDevice*. Перед первым вызовом *cudaMemcpyAsync* происходит фиксирование времени начала передач $start_t$. Таким образом, постепенно инициализируются асинхронные передачи от всех видеокарт ко всем, но данные, предназначенные для видеокарт на других процессах, требуют передачи посредством MPI. Передачи с помощью MPI также будут производиться асинхронно через *MPI_Isend*, *MPI_Irecv*. После асинхронной отправки всех сообщений и помещения запросов на прием в MPI-коммутатор все процессы будут ожидать получения какого-либо сообщения от процесса при помощи *MPI_Waitany*. Однако, данные сообщения являются сообщениями от какой-то GPU на одном узле до какой-то GPU на другом узле. Таким образом, требуется различать сообщения от разных GPU для разных GPU. Для этого используется *MPI_Tag*. Он представляет собой 32-битное целочисленное число. Отведем первые 8 бит на глобальный ранг GPU-источника и вторые 8 бит — на глобальный ранг GPU-приемника. Таким образом, после получения сообщения, будет произведена передача полученных данных от GPU под номером i посредством MPI на GPU под номером j . Однако, при приеме сообщения посредником будет выступать CPU. Декодируя *MPI_Tag* полученного сообщения, будут получены нужные номера i и j , и посредством *cudaMemcpyAsync* будет инициализировано асинхронное копирование с CPU на GPU в одном из ранее созданных на GPU потоков передачи данных. Вызов *cudaStreamSynchronize* позволяет узнать о том, что процесс копирования в выбранном потоке завершен и, в итоге, можно получить конечное время передачи

$stop_t^{ij}$. Таким образом, время передачи сообщения будет равно $stop_t^{ij} - start_t$.

5. Результаты экспериментального исследования. Подход, предлагаемый в данной работе, был апробирован на кластерной вычислительной системе, развернутой в ИПМ им. М. В. Келдыша РАН. Для этого реализованным приложением была протестирована гибридная секция суперкомпьютера K60, состоящая из 8 одинаковых узлов. Проведение данного тестирования было чрезвычайно важно для администраторов K60, так как в текущий момент времени происходит настройка и расширение данной системы.

Каждый узел гибридной секции K60 обладает следующими характеристиками [17]:

- процессоры: 2× Intel Xeon Gold 6142 v4(16× ядер);
- GPU: 4× nVidia Volta GV100GL;
- оперативная память: 768GB;
- 2 сетевых карты Gigabit Ethernet;
- 2 сетевых карты Mellanox 56G Infiniband.

Для передачи данных между узлами используется оптическая сеть Mellanox 56G Infiniband. Запуск всех тестов производился на одних и тех же узлах (использовалось 5 узлов). При запуске было соблюдено ограничение на назначение одного MPI-процесса на один узел. Таким образом, производилось тестирование передачи информации между видеокартами через шину и сеть суперкомпьютера.

Версии используемых библиотек и реализации стандартов:

- MPI – OpenMPI, версия 4.0.1;
- CUDA – версия 10.0;
- NetCDF – версия 4.7.0.

Постановка теста one to one в очередь задач суперкомпьютера производилась посредством команды:

```
mpirun -resource gpu=4 -np 5 -ppn 1 ./network_test2
-t one_to_one_cuda -b 0 -e 10000 -s 100 -n 100
```

Постановка теста all to all в очередь производилась с помощью команды:

```
mpirun -resource gpu=4 -np 5 -ppn 1 ./network_test2
-t all_to_all_cuda -b 0 -e 10000 -s 100 -n 100
```

Таким образом, общими параметрами для тестирования являются:

- начальная длина сообщения: 0 байт;
- конечная длина сообщения: 10000 байт;
- шаг изменения длины сообщения: 100 байт;
- количество итераций: 100.

При постановке задачи в очередь система очередей дает доступ ко всем GPU на узле. С помощью matplotlib были построены следующие отображения матриц задержек в виде изображений в градациях серого (см. рис. 4 и 5).

На рис. 4 отображены размеры медиан задержек для длины сообщений 0, 5000 и 9900 байт в режиме one to one. Вдоль оси X расположены номера GPU-источников в системе, вдоль оси Y номера GPU-приемников, интенсивность цвета показывает величину задержки. Видно, что порядок

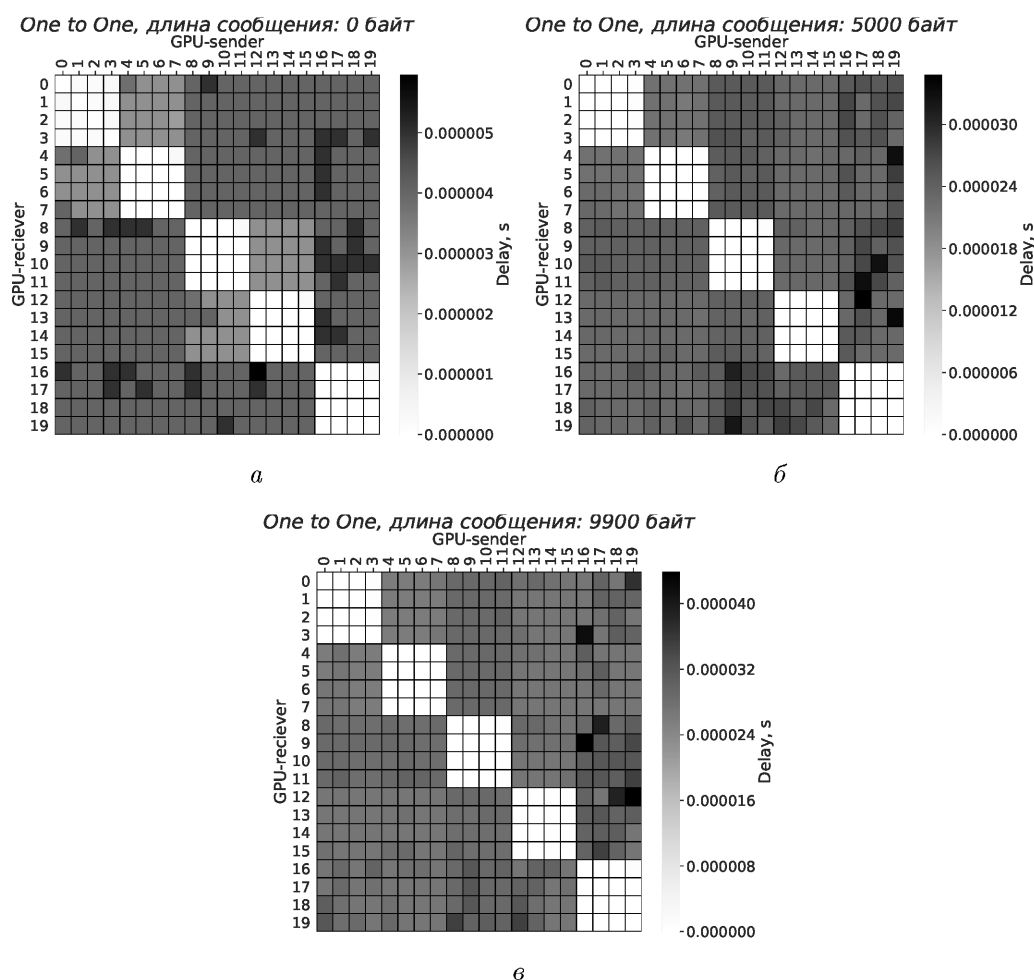


Рис. 4. Результаты тестирования K60 one to one для разных длин сообщений

задержек различен для разных размеров сообщений. В целом, задержки являются однородными, однако для GPU, расположенных на последнем узле, порядок задержек варьируется. Также очевидно, что передача информации между GPU на одном узле занимает несущественное время относительно случая передачи данных через сеть суперкомпьютера.

На рис. 5 отображены размеры медиан задержек для длины сообщений 0, 5000 и 9900 байт в режиме all to all. Вдоль оси X расположены номера GPU-источников в системе, вдоль оси Y номера GPU-приемников, интенсивность цвета показывает величину задержки. Здесь также видно, что порядок задержек изменяется с увеличением длины сообщения. Однако, общая картина отличается от режима one to one. На примере сообщения размером 0 байт видно, что наибольшее время заняли передачи от видеокарт, расположенных на первом узле. Также наибольшая задержка получилась при отправке данных с последних двух GPU первого узла. Заметим, что для других узлов неоднородностью величины задержки можно пренебречь. Но для сообщений размером 5000 байт и 9900 байт, видно, что наибольшей величиной задержки характеризуются GPU, расположенные на 3 узле. Возможно, это связано с настройкой алгоритмов управления перегрузкой на суперкомпьютере. Причем, больше всего времени было занято в процессе передачи информации на первую GPU каждого узла. Это может быть связано с приоритизацией потоков-передач с CPU на GPU.

6. Выводы и дальнейшая работа. После построения матриц задержек для разных режимов и для разной длины сообщений стала явной неоднородность сети и передач между GPU. Размер задержек варьируется в зависимости от того, какой порядок назначен GPU на узле и на каком узле она расположена. Для GPU на последнем узле стало видно, что передачи занимают

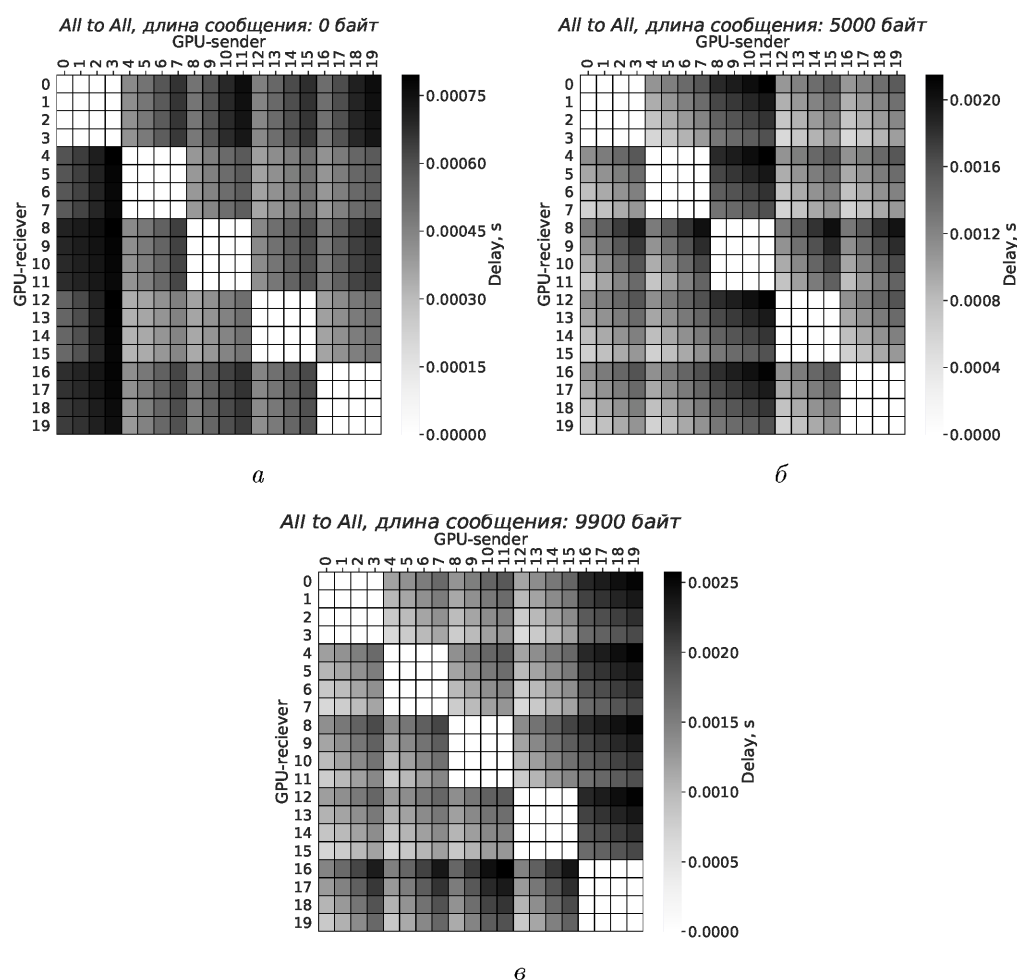


Рис. 5. Результаты тестирования K60 all to all для разных длин сообщений

большее время, чем для остальных узлов и GPU. Таким образом, при распределении задачи с распараллеливанием на GPU, следует учитывать данный эффект. Для более эффективного распределения вычислений следует решать задачу мапинга: правильно выбирать GPU и узлы, на которых расположены вычисления.

В дальнейшем целесообразно рассмотреть другие схемы тестирования. Одной из таких схем может быть часто используемый принцип, когда в системе задействована только одна GPU, а остальные MPI-процессы обмениваются с ней данными. Также собираемые данные могут быть подвергнуты анализу, подобному изложенному в статье [19], с возможностью применения различных алгоритмов интеллектуального анализа данных, например, кластеризации. Выработать схему предоставления рекомендаций для распределения задач и выявления неполадок в вычислительном кластере.

СПИСОК ЛИТЕРАТУРЫ

1. Беляев К. П., Кулешов А. А., Тучкова Н. П. Моделирование динамики океана с усвоением данных наблюдений // Препринты ИПМ им. М. В. Келдыша. 2018. № 133. С. 1–13.
2. Zou S. X., Chen C. Y., Wu J. L., Chou C. N., Tsao C. C., Tung K. C., Chang E. Y. Distributed training large-scale deep architecture // International Conference on Advanced Data Mining and Applications. Cham: Springer, 2017. P. 18–32.
3. Bazilevs Yu., Takizawa K., Tezduyar Ta. New directions and challenging computations in fluid dynamics modeling with stabilized and multiscale methods // Mathematical Models and Methods in Applied Sciences. 2015. **25**. P. 2217–2226.

4. NVIDIA Tesla P100 [Электронный ресурс]. URL: <https://images.nvidia.com/content/pdf/tesla/whitepaper/pascal-architecture-whitepaper.pdf> (дата обр. 05.05.2019).
5. Описание суперкомпьютера Summit. URL: <https://www.olcf.ornl.gov/olcf-resources/compute-systems/summit> (дата обр. 2.07.19).
6. Perepelkina A., Levchenko V. LRnLA algorithm conefold with non-local vectorization for LBM implementation // Supercomputing. RuSCDays 2018. Communications in Computer and Information Science. Vol. 965. Cham: Springer, 2018. P. 101–113.
7. Colmenares J., Galizia A., Ortiz J., Clematis A., Rocchia W. A combined MPI-CUDA parallel solution of linear and nonlinear Poisson-Boltzmann equation // Biomed Res Int. Vol. 2014. 2014. Article ID 560987. P. 1–12.
8. MPI Standart [Электронный ресурс]. URL: <https://www.mpi-forum.org/docs/mpi-3.1/mpi31-report.pdf> (дата обр. 14.05.2019).
9. CUDA Developer Documentation [Электронный ресурс]. URL: <https://docs.nvidia.com/cuda/> (дата обр. 12.05.2019).
10. Сальников А. Н., Андреев Д. Ю., Лебедев Р. Д. Инструментальная система для анализа характеристик коммутационной среды вычислительного кластера на основе функций MPI // Вестн. Моск. ун-та. Сер. 15. Вычисл. матем. и киберн. 2012. № 1. С. 39–48. (Salnikov A. N., Andreev D. Y., Lebedev R. D. Toolkit for analyzing the communication environment characteristics of a computational cluster based on mpi standard functions // Moscow Univ. Computat. Math. Cybern. 2012. 36. N 1. P. 41–49.)
11. Fujiwara T. et al. A visual analytics system for optimizing the performance of large-scale networks in supercomputing systems // Visual Informatics. Vol. 2. Zhejiang, China, 2018. P. 98–110.
12. Bhatele A., Laxmikanth V. An evaluative study on the effect of contention on message latencies in large supercomputers. // 2009 IEEE International Symposium on Parallel & Distributed Processing. IEEE. 2009. P. 1–8.
13. Pennycook S. J., Hammond S. D., Mudalige G. R., Jarvis S. A. Performance analysis of a hybrid MPI/CUDA implementation of the NAS-LU benchmark // ACM SIGMETRICS Performance Evaluation Review-Measurement and Evaluation. 2011. 38(4). 23. P. 23–29.
14. Clustbench [Электронный ресурс]. URL: <https://github.com/clustbench> (дата обр. 03.05.2019).
15. NetCDF4 Documentation [Электронный ресурс]. URL: <https://www.unidata.ucar.edu/software/netcdf/docs/index.html> (дата обр. 09.05.2019).
16. Li H., Yu D., Kumar A., Tu Y. C. Performance modeling in CUDA streams – a means for high-throughput data processing // Proc IEEE Int. Conf. Big Data. IEEE, 2014. P. 301–310.
17. Вычислительный комплекс К-60 [Электронный ресурс]. URL: <http://www.kiam.ru/MVS/resources/k60.html> (дата обр. 13.05.2019).
18. Pacheco P. Parallel Programming with MPI. US: Morgan Kaufmann, 1996.
19. Salnikov A. N., Begaev A. A., Maysuradze A. I. Analysis of delays structure of interconnections in supercomputer by means of dbscan and divisive clustering algorithms // Computational Mathematics and Information Technologies. 2018. 2. N 1. P. 33–43.

Поступила в редакцию 09.07.19

После доработки 02.10.19

Принята к публикации 02.10.19